

Tutorial Week 3

Think about the following scenario:

- You're a business owner and you want to utilize the power of LLM to create an intelligent customer service agent that can answer questions about your company's business.
- You are a programmer, and you want the Big Model to be able to look at API documentation and give accurate answers, rather than making up non-existent APIs.
- You are a student, you hope that when you ask a question to the big model, the model can not only give an answer, but also correspond to the specific knowledge points in the textbook.

However, the existing cutting-edge LLMs are pre-trained models, and the knowledge they have is limited to the training corpus, which is not able to solve the above practical needs.

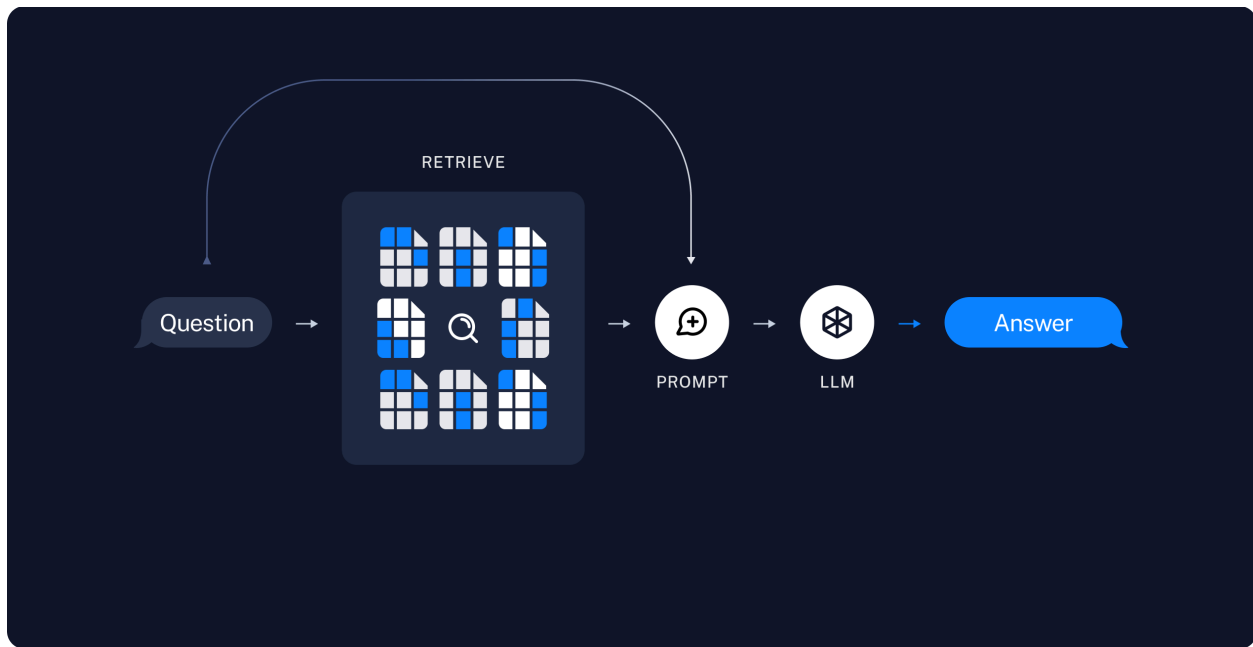
Is there a suitable solution?

When asking a question to the model, send the required documents to the model together? (Context? Efficiency? Cost? Speed of reasoning?)

RAG (Retrieve Augmented Generation)

Retrieve content from a knowledge base and then generate answers based on that content

Basic Process



1. The user uploads a document to the knowledge base and the system slices and dices the document into a series of smaller documents.
2. After the user asks a question, the system finds the small documents from the knowledge base that best match the answer.
3. The system embeds hints in these small documents as context and sends them to the model.
4. The model generates a better answer based on this information.

The RAG process can be roughly divided into the following five parts: slicing, indexing, recalling, rearranging, and generating.

Chunking

The purpose of chunking is to refine the document into smaller parts so that it can be easily sent to the model.

There are mainly the following common chunking methods:

- By word count
- By paragraph

- By chapter
- By page number
- Frontier Chunking techniques

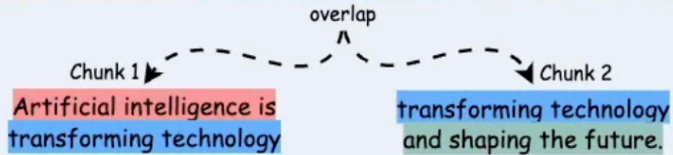
5 Chunking Strategies for RAG



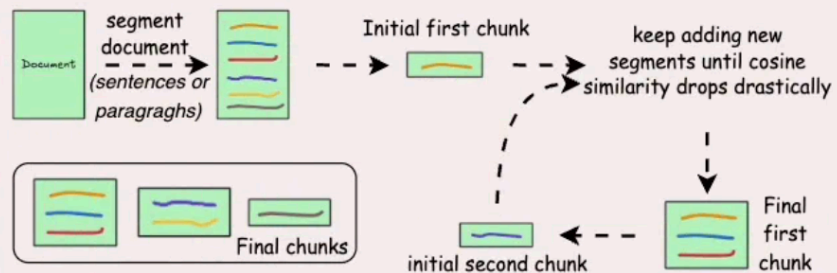
blog.DailyDoseofDS.com

1) Fixed-size chunking

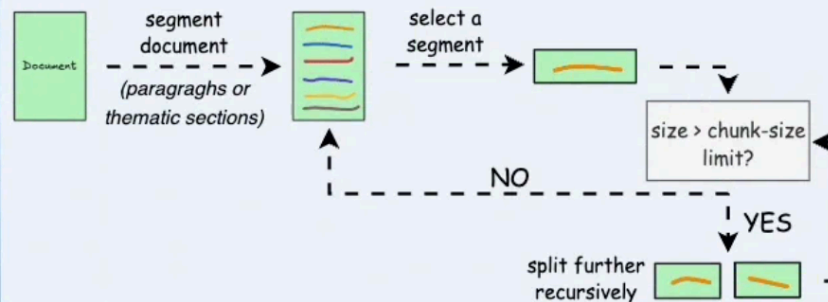
Artificial intelligence is transforming technology and shaping the future.



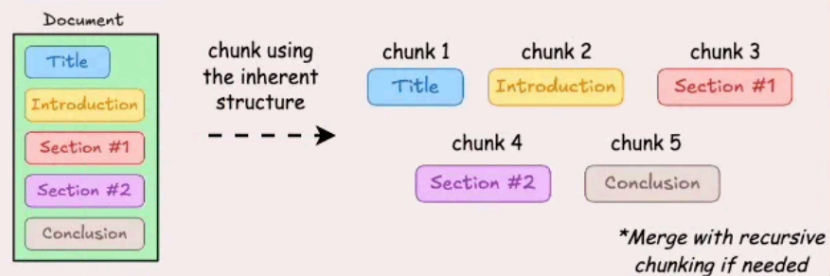
2) Semantic chunking



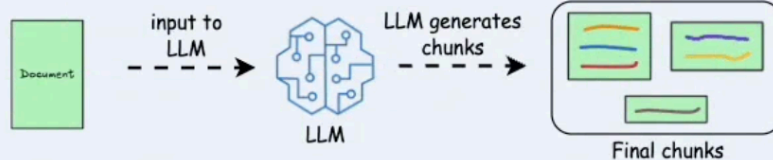
3) Recursive chunking



4) Document structure-based chunking

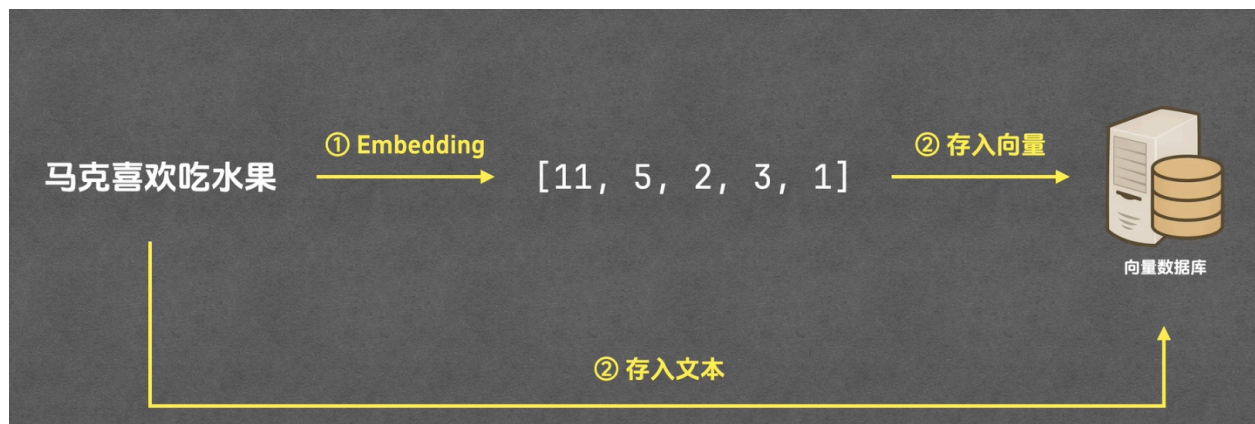


5) LLM-based chunking



Indexing

Converting chunked text into vectors by Embedding, then storing the chunked raw text and vectors in a vector database

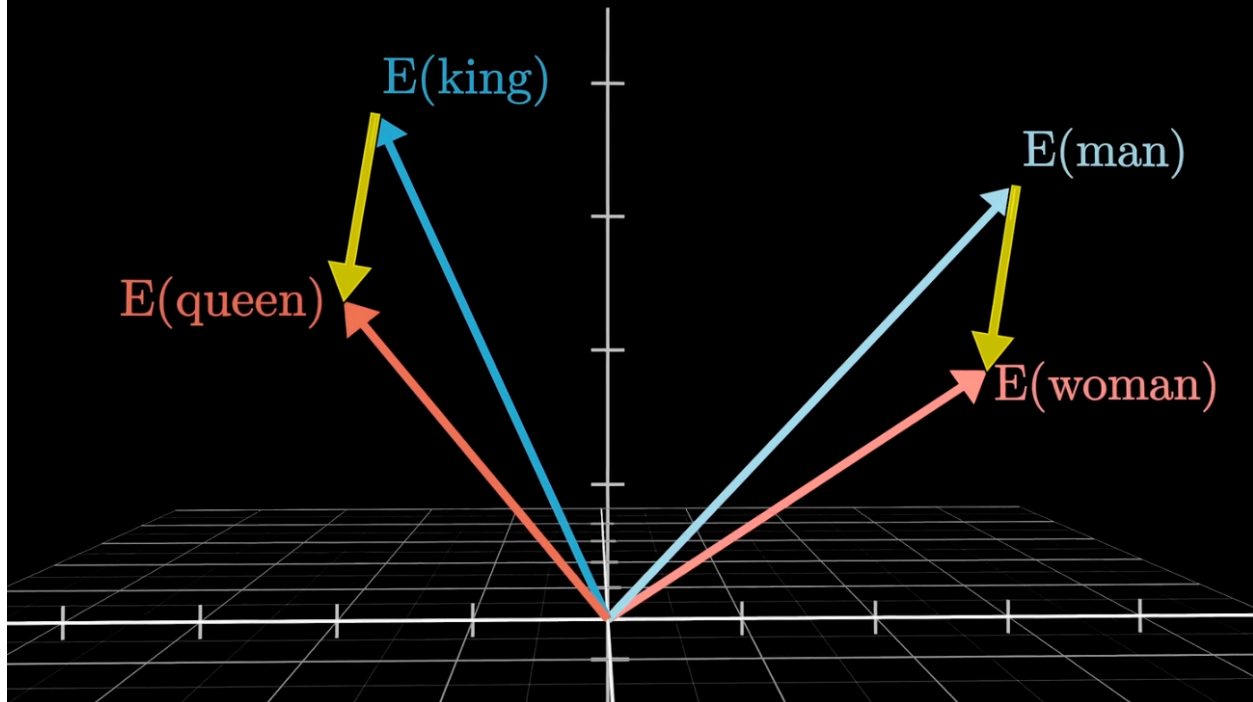


Vectors and their applications

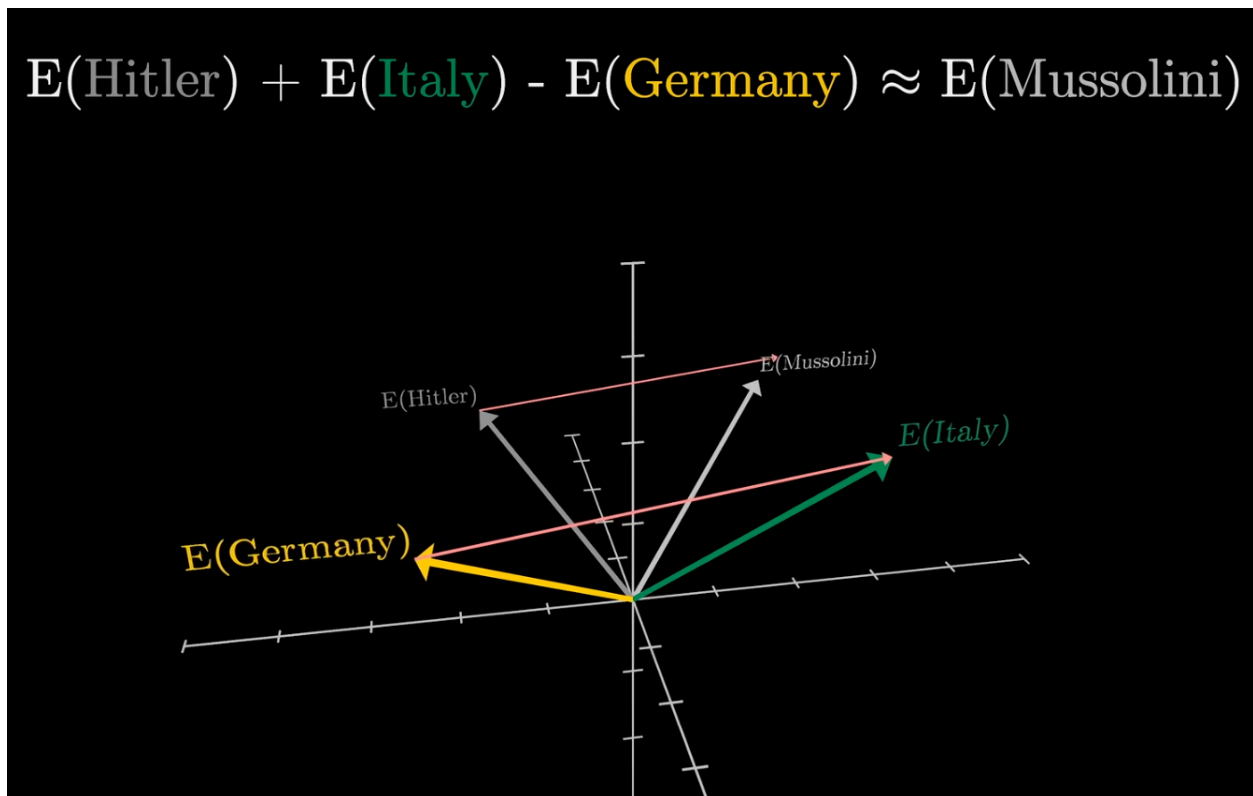
Vectors can be used to store information, and two words or sentences that are semantically close to each other have similar orientations in the vector space.

After transforming two chunks into vectors by embedding, we can judge the semantic relevance of the two chunks by the similarity between their vectors.

$$E(\text{queen}) - E(\text{king}) \approx E(\text{woman}) - E(\text{man})$$



$$E(\text{Hitler}) + E(\text{Italy}) - E(\text{Germany}) \approx E(\text{Mussolini})$$



Embedding chunks into vectors uses an embedding model, the ranking of different embedding models can be found in the following link

<https://huggingface.co/spaces/mteb/leaderboard>

Summary	Performance per Model Size	Performance per Task Type	Performance per task	Task information			
Filter...							
Rank (Bor...	Model	Zero-shot	Memory U...	Number of P...	Embedding D...	Max Tokens	Mean (T...
1	gemini-embedding-001	99%	Unknown	Unknown	3072	2048	68.37
2	Qwen3-Embedding-8B	99%	28866	7B	4096	32768	70.58
3	Qwen3-Embedding-4B	99%	15341	4B	2560	32768	69.45
4	Qwen3-Embedding-0.6B	99%	2272	595M	1024	32768	64.34
5	gte-Qwen2-7B-instruct	⚠ NA	29040	7B	3584	32768	62.51
6	Linq-Embed-Mistral	99%	13563	7B	4096	32768	61.47
7	multilingual-e5-large-instruct	99%	1068	560M	1024	514	63.22
8	embeddinggemma-300m	99%	578	307M	768	2048	61.15
9	SFR-Embedding-Mistral	96%	13563	7B	4096	32768	60.90
10	GritLM-7B	99%	13813	7B	4096	32768	60.92
11	text-multilingual-embedding-002	99%	Unknown	Unknown	768	2048	62.16

Vector Database

Databases for storing and querying vectors, optimized for storing vectors, and supporting functions to compute the similarity of vectors.

Generally, a vector database table has at least two columns, the original text and the corresponding vectors.

Recall

Search for chunks related to the user's question

- The user asks a question and the Embedding model embeds it into a vector.
- Send the embedded vectors to the vector database and query the ten most similar vectors.
- Return the original text corresponding to the ten most similar vectors.

Similarity $([11, 5, 2, 3, 1], [4.8, 3.7, 1.5, 5.2, 6.0]) = 0.7506$

Vector similarity calculation method

Cosine similarity: the smaller the angle between two vectors, the higher the similarity.

Euclidean distance: calculate the distance between two vectors, the smaller the distance, the higher the similarity.

Dot product: consider the direction and length of two vectors, the larger the dot product is, the higher the similarity is.

Re-ranking

Re-ranking is reordering, which is essentially a kind of recall.

We need to refine the ten original texts obtained from the recall, and extract the three most relevant texts from them.

Why don't we just extract the three most relevant texts in the recall phase?

The re-ranking phase uses a different logic for calculating text similarity (cross-encoder)

Splice the query and document together, feed them into the same Transformer model, and output a relevance score directly at the end. When calculating the query-document, we need to run the model once for each pair of combinations, which is expensive but highly accurate.

Query: "Who wrote The Old Man and the Sea?"

Document: "The Old Man and the Sea was written by Ernest Hemingway in 1951."

[CLS] Who wrote The Old Man and the Sea? [SEP] The Old Man and the Sea was written by Ernest Hemingway in 1951. [SEP]

After Transformer processing, the model will output a **relevance score** (e.g., 0.95, which means very relevant).

Dimension	Retrieval	Re-ranking
Objective	Find candidate documents from a large repository	Fine-grained ranking of candidate documents
Quantity	Usually tens to hundreds	Usually top 5-10
Prioritization	Fast coverage to ensure no misses	Fine screening to ensure relevance
Model complexity	Light (embedding similarity, BM25)	Heavy (cross-encoder, LLM)
Performance requirements	High speed, low latency	Allow slower, due to low document volume
Analogies	Find the needles in the haystack, first get a bunch of needles	Pick the sharpest needles from the pile.

Generation

Obtain the most relevant snippets of documentation for the user's problem and embed them in the system prompts to enhance the quality of the generation.

`You are a helpful AI assistant for an educational course. Use the following context from course materials to answer the user's question.

Course Materials Context:

`${Context}`

Conversation History:

`${historyContext}`

Guidelines:

- Answer based only on the provided course materials
- Be concise and educational
- If the context doesn't contain the answer, say you don't have enough information
- Cite specific parts of the materials when possible
- Be helpful and encouraging

User Question: \${question}

Provide a clear, educational answer:`

Practice : First RAG Demo

```
git pull #Pull latest repository
```

```
conda activate cs309
```

```
pip install ipykernel
```

```
python -m ipykernel install --user --name cs309 --display-name cs309-jupyter  
r #register kernel
```

```
pip install sentence_transformers chromadb
```